

FRONT

J1 — Setup projet

- Créer projet Flutter, structure Clean Architecture (data/domain/presentation)
- Configurer pubspec.yaml : flutter_bloc, freezed, dio, hive, flutter_map, geolocator, flutter_tts
- Permissions iOS (Info.plist) et Android (Manifest) pour GPS, caméra
- Theming Material 3 (couleurs, typographie) + configuration go_router

J2 — Design system

- Composants réutilisables : boutons, champs de saisie, cards, loaders
- Définir la structure de navigation (routes, guards)
- Mettre en place les constantes (couleurs, espacements, textes)

J3 — Écrans Auth (UI seule)

- SplashScreen (logo + vérification token à venir)
- LoginScreen, RegisterScreen, ForgotPasswordScreen — formulaires + validation front, sans appel API encore

J4 — Auth fonctionnel (dépend Back J3-J4)

- AuthBloc (états : loading, success, error) + AuthRepository
- Intercepteurs Dio (ajout token auto, refresh automatique sur 401)
- Connexion réelle des écrans Auth aux endpoints
- Stockage sécurisé des tokens (Hive + FlutterSecureStorage)

J5 — Géolocalisation

- LocationService : demande permissions, stream de position en continu
- Filtrage basique des positions aberrantes côté client
- Test sur device réel (précision, fréquence de mise à jour)

J6 — Carte principale

- MapBloc + HomeScreen : affichage FlutterMap avec tuiles OSM

- Marqueur position GPS en temps réel
- Sélecteur de mode de déplacement (piéton/moto/voiture), persisté localement

J7 — Recherche destination (⌚ dépend IA J5 — Nominatim déployé)

- SearchScreen : barre de recherche avec debounce 300ms
- Appel /navigation/autocomplete, affichage suggestions
- Historique des recherches (Hive), sélection → transition vers RoutePreview

J8 — Aperçu itinéraires (⌚ dépend IA J6 — endpoint route prêt)

- NavigationBloc + NavigationRepository
- RoutePreviewScreen : affichage 2 polygones (rapide/court), métriques (distance, durée)
- Badge "Optimal" si un itinéraire combine les deux, bouton Démarrer

J9 — Navigation temps réel (setup) (⌚ dépend Back J9 — WebSocket prêt)

- NavigationScreen : carte orientée selon le cap (heading)
- TrackingService : connexion WebSocket, gestion reconnexion auto
- Buffer local Hive en cas de perte de connexion

J10 — Guidage vocal & finalisation navigation

- Intégration flutter_tts : annonces à 500m/200m/30m avant le tournant
- Barre de progression (waypoints parcourus en gris, restants en bleu)
- Détection arrivée (<50m) → TripSummaryScreen
- Déclenchement recalcul si écart >40m pendant 5s

J11 — Intégration Gateway

- Basculer toutes les requêtes API vers Kong (baseUrl)
- Vérifier propagation correcte des tokens JWT via Kong
- Tests de bout en bout sur les routes protégées

J12 — Tests bout en bout

- Scénario complet sur device Android physique + simulateur iOS
- Tests des cas d'erreur (pas de réseau, token expiré, GPS désactivé)
- Remontée et priorisation des bugs (P0/P1)

J13 — Polissage UX

- Skeleton loaders sur les écrans de chargement
- Messages d'erreur explicites et traduits
- Gestion "pas de connexion" (connectivity_plus)
- Correction des rebuilds inutiles via Flutter DevTools

J14 — Documentation

- README frontend (setup, lancement, structure des dossiers)
- Captures d'écran / GIFs pour la démo

J15 — Démo

- Gel du code à 10h, corrections P0 uniquement
- Participation à la démonstration

BACK

J1 — Setup infra

- Créer le monorepo GitHub, structure de dossiers pour chaque microservice
- docker-compose.yml : PostgreSQL+PostGIS, Redis, MinIO, Mailhog, Adminer
- Pipeline CI GitHub Actions (lint + build Docker)

J2 — Schéma BDD + squelette Auth

- Migrations Alembic : tables users, routes, segments, gps_tracks
- Activation extensions PostGIS + index GIST/B-tree
- Structure auth-service (FastAPI, SQLAlchemy async, config, schemas)
- Vérification /health + seeds de développement

J3 — Endpoints Auth

- POST /auth/register (validation, bcrypt, JWT + refresh Redis)
- POST /auth/login (rate limiting 5 tentatives/15min via Redis)
- POST /auth/refresh (rotation refresh token)
- POST /auth/logout (blacklist Redis)
- GET/PATCH /users/me + middleware get_current_user

- Tests unitaires Pytest (min. 15 cas)

J4 — OAuth & récupération mot de passe

- POST /auth/google + POST /auth/apple (validation tokens tiers)
- POST /auth/forgot-password + POST /auth/reset-password (envoi email via Mailhog en dev)

J5 — Squelette Navigation Service (dépend IA J5 — OSRM/Nominatim up)

- Structure navigation-service (FastAPI)
- POST /navigation/geocode, GET /navigation/autocomplete, GET /navigation/reverse
- Mise en cache Redis des résultats fréquents

J6 — Modèles routes/segments

- Finaliser modèles SQLAlchemy pour routes et segments
- Squelette tracking-service (structure WebSocket, pas encore de logique métier)

J7 — Tracking Service

- Endpoint WebSocket /tracking/session/{route_id}
- Logique de buffer des positions → flush périodique vers PostgreSQL (format LINESTRING)

J8 — Traitement asynchrone (coordination avec IA J8)

- Mise en place de la queue Celery (broker Redis)
- Tâche process_completed_trip : appelée en fin de trajet, prépare les données pour l'upsert des segments

J9 — Finalisation Tracking (→ débloque Front J9)

- Finaliser calcul des métriques de fin de trajet (distance, durée, vitesse moy.)
- Déclenchement automatique de la tâche Celery en fin de session WebSocket
- Tests de charge basique sur le WebSocket

J10 — Support performance

- Optimisation des requêtes PostGIS utilisées pendant la navigation active
- Vérification de la latence de /navigation/route en conditions réelles

J11 — Kong Gateway

- Déclaration de tous les services et routes dans Kong

- Plugins : jwt, rate-limiting (sur auth), cors, response-transformer
- Health checks Kong vers chaque microservice

J12 — Tests d'intégration

- Scénario complet : Register → Login → Geocode → Route → WebSocket → vérif BDD → Refresh → Logout
- Scénario rate limiting (6 tentatives → 429)
- Correction bugs P0/P1

J13 — Optimisations

- EXPLAIN ANALYZE sur /navigation/route, identifier les lenteurs
- Vérification hit rate cache Redis (header X-Cache)
- Nettoyage des logs (suppression SQL verbeux, données sensibles)

J14 — Documentation

- README complet (prérequis, démarrage en 5 étapes)
- Enrichissement Swagger (exemples, codes d'erreur)
- ARCHITECTURE.md (décisions techniques)

J15 — Démo

- Gel du code, corrections P0
- Support technique pendant la démonstration



IA

J1 — Préparation données

- Lancer tôt le téléchargement des données OSM de la zone pilote (.osm.pbf) — étape longue
- Setup environnement Python : NetworkX, OSMnx, pykalman, NumPy

J2 — Modélisation du graphe

- Construction du graphe pondéré $G(V,E,W)$ à partir des données OSM
- Définition de la fonction de poids composite (distance, temps observé, vitesse par mode)

J3 — Prototypage algorithmes

- Implémentation et tests Dijkstra + A* sur graphe local (NetworkX)
- Comparaison des performances sur petits/grands graphes

J4 — Filtrage GPS

- Prototypage du filtre de Kalman pour lisser les positions GPS aberrantes
- Tests sur données GPS simulées (canyons urbains, sauts de position)

J5 — Déploiement moteur de routage (→ débloque Back J5)

- Déploiement OSRM (extract → partition → customize → routed) en Docker
- Déploiement Nominatim + import complet des données OSM de la zone pilote
- Vérification des deux services (requêtes test)

J6 — Logique de calcul d'itinéraire (⏳ dépend Back J5)

- Implémentation de la logique /navigation/route :
 - Vérification couverture segments SolRoad (PostGIS)
 - Calcul via A*/Dijkstra si couverture suffisante
 - Fallback automatique sur OSRM sinon
- Tests de performance (objectif P95 < 2s)

J7 — Intégration Kalman en production

- Intégration du filtre de Kalman dans le flux de données GPS entrant (temps réel)
- Ajustement des paramètres selon les tests terrain (coordination avec Front J5)

J8 — Traitement post-trajet (⏳ dépend Back J7-J8)

- Finalisation tâche Celery process_completed_trip :
 - Extraction des segments parcourus
 - Upsert dans la table segments (mise à jour sample_count)
 - Calcul initial du confidence_score

J9 — Route Discovery Job

- Mise en place de Celery Beat (exécution toutes les 15 min)
- Recherche de segments adjacents (PostGIS ST_DWithin, rayon 50m)
- Logique de composition (Route Stitching) des nouveaux itinéraires potentiels

J10 — Recalcul dynamique

- Logique de snap-to-road (projection position sur segment le plus proche)
- Déclenchement du recalcul si écart >40m pendant 5s (appel à /navigation/route)

J11 — Tests sous Gateway

- Vérifier que le passage par Kong n'impacte pas la performance du calcul d'itinéraire

J12 — Tests Route Discovery

- Scénario : 3 segments adjacents connus → vérifier qu'une route composée est bien créée et enregistrée

J13 — Optimisation algos

- Profiling du graphe (temps de calcul, mémoire)
- Ajustement de la concurrence des workers Celery

J14 — Documentation

- Documentation des algorithmes utilisés et des choix techniques (pourquoi Dijkstra/A*/OSRM, paramétrage Kalman)

J15 — Démo

- Gel du code, corrections P0
- Support technique pendant la démonstration

Dépendances critiques (points de vigilance)

Jour	Dépendance	Risque si retard
J5	IA (OSRM/Nominatim) → Back (endpoints geocode)	Bloque la recherche d'adresse
J5- J6	Back (squelette navigation) → IA (logique route)	Bloque le calcul d'itinéraire
J6	IA (route prête) → Front (RoutePreview)	Bloque l'affichage des itinéraires
J7- J8	Back (Celery/tracking) → IA (traitement post-trajet)	Bloque l'enrichissement des segments
J9	Back (WebSocket tracking) → Front (navigation temps réel)	Bloque la navigation live

Jour Dépendance**Risque si retard**

J3-
J4 Back (endpoints Auth) → Front (Auth fonctionnel) Bloque la connexion utilisateur

Conseil : un point de sync quotidien de 10 min entre les 3 équipes, avec attention particulière autour des J5-J6 et J8-J9 (points de convergence les plus sensibles du planning).